

StencilMaker demo

Purpose

The purpose of this program is to simplify stencil generation for single component packages. Specifically, it generates an STL file of the designed stencil, making it ready for 3D printing.

A brief usage summary

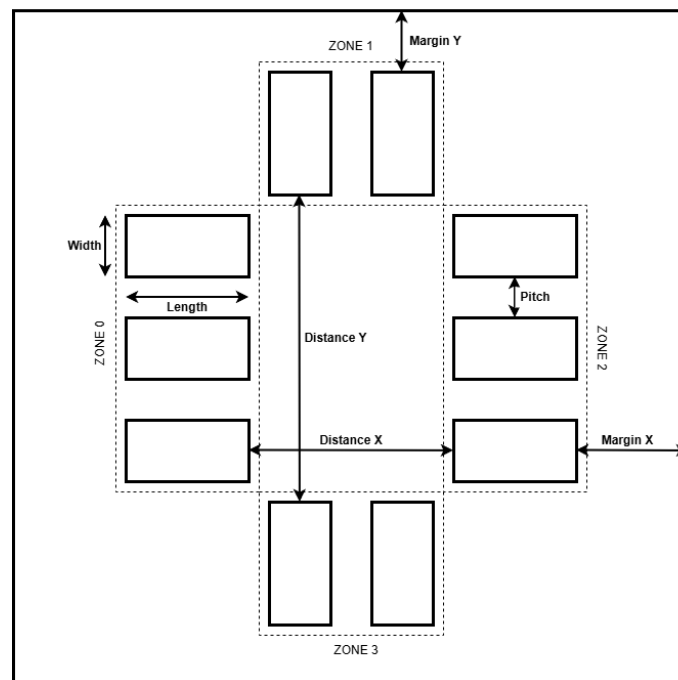
First, select the desired stencil type. In this demo, two of the most common package types are available: *DIP* and *QFP*. Once the type is selected, you can either use a preset package or create a custom one. If you choose a preset, certain fields will automatically populate, and you will only need to input the specific parameters for the stencil itself, such as height, margins, and tolerance.

Note: Before filling out any fields, you must select the measurement unit. The default unit is millimeters.

Once all fields are filled, click the *Run* button to generate the stencil (if no error occurs). Use the *Folder* button to select the destination directory for the STL file, and then press the *Save* button to export the model.

GUI description

Tab 'Design':



a. Stencil diagram.

Input fields

The stencil is defined by fourteen parameters, as highlighted in the diagram above. Naturally, certain parameters do not apply to specific package types (for example, the *pin distance y* is not used for DIP packages). To prevent unnecessary data entry, all input fields are disabled when the program opens and are unlocked only after a package type is selected.

Note: The program only accepts a period as a decimal separator. Using any other character will trigger an "invalid value" error in the log window. Moreover, only three decimal digits are shown.

Unit selector

The unit selector allows you to input and view data in your preferred unit of measurement. Changing this setting updates not only how the program interprets your inputs, but also how it displays pre-loaded package data and the limit values in the second tab. To maintain data integrity, the unit selector will be locked as soon as any value is entered into the input fields or a preset package is loaded. It will become available again only after clearing all input fields.

Log window

The log window displays real-time progress and errors during stencil generation. To assist with debugging unexpected behavior, all messages shown here are simultaneously written to a log file stored in the *log_files* folder.

Control Console

The right column of the first tab is the *Control Console*. The first two elements are the *type menu* and the *package menu*. The package menu displays options stored in text files within the corresponding type folder.

There are two ways to add new package data:

1. **Manually via Text File:** Create a new text file in the appropriate type folder, mirroring the layout of an existing file. The text preceding each value does not matter; the program only reads the numbers following the = character and identifies them based on their line order. Data in these files must be written in centimeters. The filename will define the package name in the GUI.
2. **Via the create button:** Fill out all the input fields for a custom package. Once the program verifies data integrity, the *create* button will be enabled. Clicking it opens a pop-up where you can name the package and finalize the file creation.

Below is a description of all the buttons present in the console:

1. **Folder:** Selects the saving directory (available at any time).
2. **Run:** Generates the STL file (available only when all data is valid and error-free).
3. **Save:** Saves the generated file. This button is enabled only after a folder is selected and the stencil has been successfully generated via the *Run* button. Once clicked, the *Save* button is disabled again until you click *Run* once more.
4. **Activate log file writing (Checkbox):** Enables log file generation for the current session. At startup, a log file named with the current date and time is created. If unchecked, data writing stops, though the GUI log window will continue to show messages.
5. **Clear:** Clears the GUI log window text without affecting the physical log file.
6. **Cancel:** Resets the package menu and clears all input fields.

Tab 'Options':

Limit values

To enforce data validation, the program references a list of minimum and maximum values for each data point, located in a file named *min_max_values* within the program directory. Because these limits depend on your specific 3D printer or external factors, they can be edited in two ways:

1. **Direct File Editing:** Edit the *min_max_values* text file directly, ensuring all entries are in centimeters.
2. **Via the GUI:** Use the *Limit values* section in the second tab. This section displays the current operational limits, which match the unit chosen by the *Unit Selector*. When you modify these values, the *Update* button becomes active. Clicking it updates the *min_max_values* file. You can use the *Back* button to revert to the previous values before the last update.

Saving Info

This section displays exporting details, including the current destination directory, the file name, and the export unit.

1. **File Name Validation:** You can define the file name here. The program automatically checks for characters forbidden by Windows, an empty field, or an excessive name length, disabling the Save button if any issue is found. It also checks for existing files in the target folder and will prompt you with an overwrite confirmation pop-up if a duplicate name is found.
2. **Export Unit Selector:** Allows you to choose the unit for the exported file. Standard STL files do not include tags specifying their measurement unit, though most 3D printing slicing software defaults to interpreting them in millimeters. To maintain compatibility with various external software conventions, this selector gives you full control over the output unit.

Validation function

Once all data is entered, a validation function checks the inputs against a set of rules specific to the selected package type. If any rule is broken, the STL generation is blocked, the invalid fields are highlighted in red, and an error message is displayed in the log window. The applied validation rules are outlined below:

1. **Limit Values:** Verifies that all inputs fall within the allowed minimum and maximum ranges. If any value exceeds these boundaries, an error is raised.
2. **Pin Count (DIP Only):** Confirms that the total number of pins is an even number.
3. **Tolerance:** Ensures that the tolerance value does not excessively reduce other physical dimensions, which would result in an invalid geometry.
4. **Minimum distance x and y (QFP Only):** In QFP packages, the minimum allowable values for *distance x* and *distance y* depend on the number of pins on each side. The limit is calculated using the following formula:

$$dist_y_min = more_pin_n_hor \cdot pin_width + (more_pin_n_hor - 1) \cdot pin_pitch$$

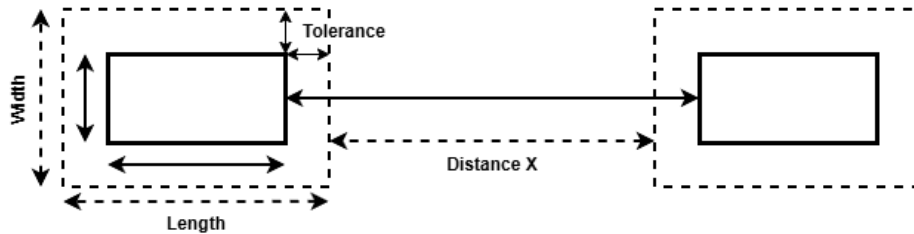
The same formula applies for the distance x, where the *more_pin_number* value considers only the vertical pins.

Tolerance

Tolerance adjusts specific stencil dimensions to compensate for the physical geometric uncertainties inherent in 3D printing. Specifically, the tolerance value modifies the parameters as follows:

1. $pin_length \ += \ 2 \cdot tolerance$
2. $pin_width \ += \ 2 \cdot tolerance$
3. $pin_pitch \ -= \ 2 \cdot tolerance$
4. $distance_x \ -= \ 2 \cdot tolerance$
5. $distance_y \ -= \ 2 \cdot tolerance$

Note: tolerance must be a non-negative value.

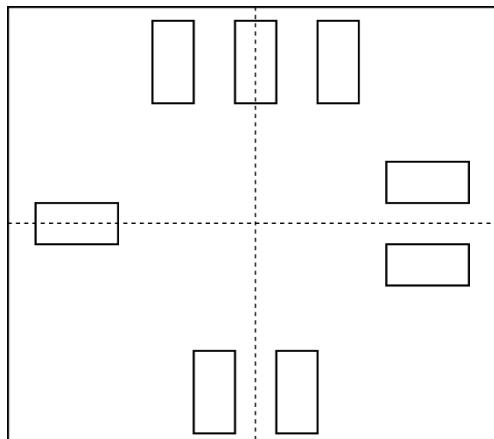


b. Tolerance effects on certain measures. The dashed ones are affected by the tolerance.

Consequently, excessively high tolerance values will drastically reduce dimensions like the pin pitch. If these values fall below the allowed lower limit, the data validation function will flag them, invalidating the model.

QFP generation

The QFP stencil generation is the most versatile feature of the program, as it allows the user to specify the pin count for each side independently. Because the ‘even pin count’ rule does not apply to QFP packages, you are completely free to customize the pin distribution. In fact, this flexibility allows the QFP generation tool to be used as a workaround to generate an asymmetric DIP package (a DIP with a different number of pins on each side) simply by setting the pin count to zero on two opposite sides. Note that if two opposite sides have a different pin count, centering the layout for the remaining two sides becomes non-trivial. To handle this, the algorithm is designed to always center the pin group along the edge, as illustrated in the figure below.



c. Pin alignment on a non-symmetrical QFP.